Attack & Detection Report

sarten home SOC lab

Author: asynthe Date: 2026-10-11 Classification: Lab / Portfolio

Executive summary

This report documents a controlled offensive exercise against the author's own home lab, carried out to validate the detection coverage of a self-hosted Wazuh SIEM. A Parrot Security workstation was used to run reconnaissance, a credential-access attack, and web-application attacks against intentionally exposed services. Each action was correlated against the alerts it produced in Wazuh, mapped to MITRE ATT&CK, and assessed for detection quality.

All targets are the author's own systems on an isolated home network; all addresses shown are private (RFC 1918). The exercise is a lab simulation, not an engagement against a third party.

Headline result — *[pending — fill after running this phase]*

Scope & environment

The lab is a single Proxmox VE host running unprivileged LXC guests, each with one Docker stack. Wazuh agents on the host and guests ship logs to a single-node Wazuh manager/indexer/dashboard. Two services are deliberately reachable from the internet through Tailscale Funnel: Jellyfin and an nginx honeypot.

Host	Address	Role	Agent
sarten	192.168.1.135	Proxmox VE host (Debian 13)	001
media	192.168.1.140	Jellyfin + arr stack	003
wazuh	192.168.1.142	Wazuh manager / indexer / dashboard	000
honeypot	192.168.1.143	nginx decoy login page	004
parrot	192.168.1.99	**Attacker** — Parrot Security 7.4	—

Table 1: Targets in scope

Wazuh version 4.14.8. The dashboard was cleared of prior events immediately before this exercise, so every alert shown here is attributable to the campaign.

Methodology & tooling

Attacks progress along the ATT&CK kill chain: reconnaissance, then credential access, then exploitation of the web tier. Tools are those shipped with Parrot Security: `nmap`, `hydra`, `nikto`, and `sqlmap`. For each technique the report records the exact command, the resulting Wazuh alert(s), and an assessment of whether and how quickly the activity was detected.

Attack narrative

Phase 1 — Reconnaissance

MITRE: `T1595` Active Scanning, `T1046` Network Service Discovery

Objective. Enumerate live hosts and fingerprint exposed services.

Command (from parrot):

```
sudo nmap -sS -sV -p 22,80,443,8096,8989,9090,3000 192.168.1.135 192.168.1.140
192.168.1.143
```

Result. The scan completed in 12 seconds and fingerprinted only intended services — no unexpected ports were open.

Host	Open	Service (`nmap -sV`)
.135 sarten	22	OpenSSH 10.0p2 (Debian)
.140 media	22, 3000, 8096, 8989	SSH; Jellyfin and Sonarr on Kestrel (.NET); port 3000 rejected probes with "Host validation failed"
.143 honeypot	22, 80	SSH; nginx

Wazuh detection. Each of nmap's `-sV` HTTP probes against the honeypot's port 80 became a honeypot alert:

Rule	Level	Count	Detail
100110	6	9	Probe paths `/`, `/nmaplowercheck…`, `/sdk`, `/evox/about`, `/HNAP1`; user-agent "Nmap Scripting Engine"

The scan-rate rule 100111 did **not** fire — `-sV` sent only 9 requests, below its threshold of 20 per minute. Scanning of SSH and the closed ports produced **no** alerts at all.



Figure 1: Reconnaissance activity in the Wazuh dashboard

Analysis. The honeypot reliably flags service-enumeration traffic, and nmap did little to hide: the "Nmap Scripting Engine" user-agent and probe paths such as `/HNAP1` and `/evox/about` are unambiguous scanner fingerprints. Two gaps are worth recording. First, a low-and-slow service scan stays under the rate-based rule 100111, so the individual 100110 events carry the detection — a rule keyed on the scanner user-agent would escalate severity at once. Second, network-layer scanning of the host (SYN scans, closed ports, SSH banner grabbing) left no application log and so raised no alert; closing that gap needs a network IDS such as Suricata, tracked as a planned control.

Phase 2 — Credential access (SSH brute force)

MITRE: `T1110.001` Brute Force: Password Guessing

Objective. Attempt to brute-force SSH on the Proxmox host.

Command (from parrot):

```
hydra -l asynthe -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.135 -t 4 -V
```

Result. *[pending — fill after running this phase]*

Wazuh detection. *[pending — fill after running this phase]*

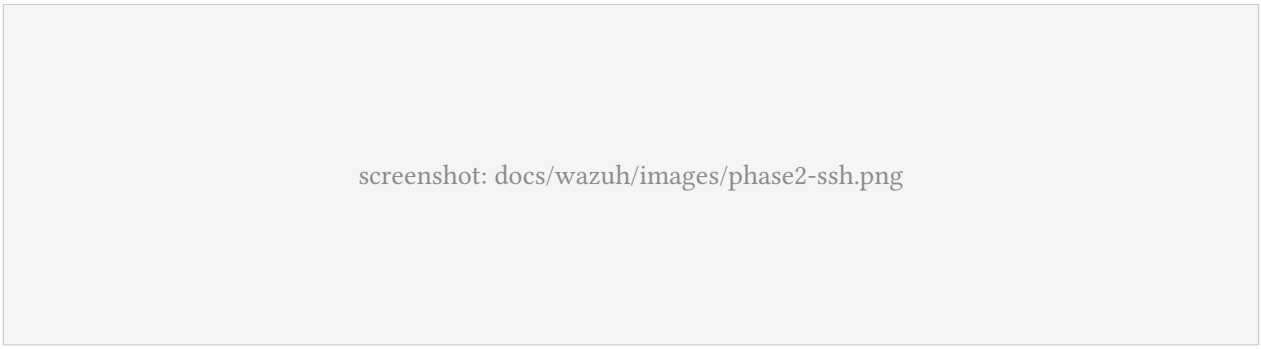


Figure 2: SSH brute-force alerts

Analysis. *[pending — fill after running this phase]*

Phase 3 — Web application attacks

MITRE: `T1190` Exploit Public-Facing Application, `T1595.002` Vulnerability Scanning

Objective. Scan and attack the exposed honeypot web service.

Commands (from parrot):

```
nikto -h http://192.168.1.143
sqlmap -u "http://192.168.1.143/product?id=1" --batch --level 2
```

Result. *[pending — fill after running this phase]*

Wazuh detection. *[pending — fill after running this phase]*



Figure 3: Web-attack alerts: custom and built-in web rules

Analysis. *[pending — fill after running this phase]*

Detection coverage

Phase	Technique	Rule(s)	Level	Detected
1	Active scanning (T1595)	100110	**6**	Partial
2	Password guessing (T1110.001)	5710/5712	**5**/**10**	*[pending — fill after running this phase]*
3	Exploit public-facing app (T1190)	31103/31104	**7**/**6**	*[pending — fill after running this phase]*
3	Vulnerability scanning (T1595.002)	31508/100111	**6**/**10**	*[pending — fill after running this phase]*

Table 2: Technique-to-detection matrix

Findings & recommendations

[pending — fill after running this phase]

Appendix A — Custom detection rules

The honeypot and Jellyfin detections used in this exercise are custom Wazuh decoders and rules maintained in the repository under `services/wazuh/config/ruleset/`.

Rule	Level	Fires on
100102	5	Jellyfin login failed, with the attacker's IP
100103	10	8 failed Jellyfin logins in 2 min from one IP (T1110)
100110	6	Any honeypot request — nothing legitimate visits it
100111	10	20 honeypot requests in a minute from one IP (T1595)